

Title: Predicting Red and White Wine Quality Using Physicochemical Properties and Machine Learning

Executive Summary

This final white paper evaluates whether wine quality can be predicted from objective physicochemical measurements. Using the UCI Wine Quality datasets for red and white Portuguese Vinho Verde wines, the project compares red and white wine chemistry, builds interpretable linear regression baselines, and trains Random Forest models for classification and regression. The final results show that wine chemistry provides useful predictive signal, especially alcohol, density, volatile acidity, chlorides, and sulfur dioxide measures, but the model should support rather than replace expert sensory evaluation.

Business Problem

Wine producers, distributors, and retailers often rely on expert tasting panels and subjective ratings to judge wine quality. While expert evaluation is valuable, it can be expensive, time consuming, and difficult to scale across large numbers of products. This project examines whether measurable physicochemical properties can predict expert wine quality ratings in a way that supports production decisions, product positioning, and consumer education. The project compares red and white wines separately, uses linear regression as a transparent baseline, and applies Random Forest models to improve predictive performance.

Background and History

Wine quality has traditionally been evaluated through sensory assessment, where trained experts judge aroma, taste, balance, body, sweetness, acidity, alcohol, and overall impression. Wine also contains measurable chemical attributes that can be tested objectively in a laboratory, including acidity, residual sugar, chlorides, sulfur dioxide, density, pH, sulphates, and alcohol. Prior research by Cortez et al. showed that data mining methods can model wine preferences using physicochemical properties. This project builds on that foundation by asking whether quality can be predicted and whether red and white wines should be interpreted separately because their chemical profiles differ.

Data Explanation

The project uses the Wine Quality dataset from the UCI Machine Learning Repository. The red wine dataset contains 1,599 observations, and the white wine dataset contains 4,898 observations, for a combined total of 6,497 wine samples. Both datasets contain the same original variables: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, and quality. The quality variable is an expert score ranging from 0 to 10. During preprocessing, wine_type was added to identify red or white wine, and quality_group was created so scores 3 through 5 were labeled low, score 6 was labeled medium, and scores 7 through 9 were labeled high. The combined data had no missing values, but 1,177 duplicate records were identified and flagged for review.

Methods

The analysis used exploratory data analysis, separate red and white baseline modeling, combined machine learning modeling, and advanced visualization. Summary statistics, quality distributions, duplicate checks, and red-versus-white comparisons were performed first. Correlation analysis identified which physicochemical variables were most strongly related to expert quality scores. Linear regression models were built separately for red and white wines as interpretable baselines. Random Forest classification then predicted low, medium, and high quality groups, while Random Forest regression predicted numeric quality scores. Visuals included density plots, PCA, a triangular correlation heatmap, model diagnostics, and alcohol-by-quality boxen plots.

Analysis

Wine quality scores were concentrated in the middle of the rating scale. Quality score 6 represented 43.65 percent of the combined dataset and quality score 5 represented 32.91 percent, while extremely high and low scores were rare. This confirms that class imbalance is an important modeling challenge. White wines had a slightly higher average quality score than red wines, with an average of 5.878 compared with 5.636 for red wines. Red wines had higher average volatile acidity, while white wines had much higher average residual sugar. Alcohol was positively associated with quality in both datasets. For red wine, the strongest quality relationships were alcohol at 0.476, volatile acidity at -0.391, sulphates at 0.251, citric acid at 0.226, and total sulfur dioxide at -0.185. For white wine, the strongest quality relationships were alcohol at 0.436, density at -0.307, chlorides at -0.210, volatile acidity at -0.195, and total sulfur dioxide at -0.175.

Modeling Results

The separate linear regression baseline models produced moderate predictive performance. The red wine linear regression model had a training RMSE of 0.652 and a test RMSE of 0.627. The white wine linear regression model had a training RMSE of 0.749 and a test RMSE of 0.757. The combined Random Forest regression model produced a test RMSE of 0.614 and an R-squared value of 0.493. The Random Forest classification model achieved an accuracy of 0.735 and a weighted F1 score of 0.735 when predicting low, medium, and high quality groups. Feature importance showed that alcohol was the most important predictor, followed by density, volatile acidity, chlorides, total sulfur dioxide, residual sugar, free sulfur dioxide, sulphates, citric acid, and pH.

Conclusion

Physicochemical wine measurements provide meaningful information about expert quality ratings. Alcohol was consistently the strongest predictor, while density, volatile acidity, chlorides, and sulfur dioxide measures also helped explain quality differences. The final project is stronger than a single-method approach because it includes separate red and white wine analysis, interpretable linear regression baselines, and a more flexible combined Random Forest model. The model should be viewed as a decision support tool for producers and retailers rather than a replacement for expert tasting.

Assumptions

This analysis assumes that the UCI red and white wine datasets accurately represent the Portuguese Vinho Verde samples described in the original study and that the expert quality score is reliable enough to serve as a modeling target. It also assumes that the available physicochemical variables capture meaningful differences in wine composition. The `quality_group` variable assumes that grouping scores into low, medium, and high categories makes the prediction task more practical for business use. Another assumption is that combining red and white wine data is acceptable when `wine_type` is included as a model feature.

Limitations

The dataset only includes Portuguese Vinho Verde wines, so the results may not generalize to other regions, grape varieties, or wine styles. The data does not include grape variety, winery, vineyard location, vintage year, climate, fermentation method, aging process, price, sales volume, or consumer review data. The quality score is based on expert evaluation, which may not reflect every consumer preference. The target variable is imbalanced, with most wines rated 5 or 6, making it harder to predict very high or very low quality wines. Duplicate records also require additional investigation before final deployment.

Challenges

The main challenges were duplicate records, class imbalance, and the decision to model red and white wines separately or together. Removing duplicate rows could reduce repeated observations, but keeping them may be reasonable if they represent legitimate wines with identical measurements. Separate models improve interpretation, while a combined model may be easier to implement. The project also had to communicate results responsibly because a model-generated prediction should not be treated as a final judgment of taste or market value.

Future Uses and Additional Applications

Future work could expand the dataset with grape variety, price, vintage, region, weather, production method, and consumer review data. This would allow the project to connect expert quality ratings with business outcomes such as sales performance or market acceptance. The model could also be tested on other wine regions to see whether the same predictors remain important. In production, the model could be used as an early quality control tool that helps winemakers identify batches that may need adjustment before bottling.

Recommendations

Producers should monitor alcohol, density, volatile acidity, chlorides, sulphates, and sulfur dioxide measures as important indicators related to quality. Linear regression should remain a baseline because it is easy to interpret, while Random Forest should be used as a stronger predictive model because it captures more complex relationships. Red and white wines should be compared separately during interpretation, even if a combined model is used operationally. Before deployment, duplicate records should be reviewed, additional validation should be performed, and performance should be evaluated by quality group rather than only through overall accuracy.

Implementation Plan

A practical implementation plan would begin by collecting new laboratory measurements from wine batches and validating that the fields match the model input structure. Each wine would receive a predicted numeric quality score and a predicted quality group. The prediction should be paired with feature importance information so producers can see which chemical properties influenced the result. A dashboard could track alcohol, acidity, density, sulfur dioxide, predicted quality, and actual tasting results over time. The model should be retrained periodically as new wine samples are collected.

Ethical Assessment

This project does not use personal customer information, but ethical considerations still matter. Expert quality ratings should not be treated as an absolute definition of quality because taste is subjective and may differ across cultures, regions, and individual consumers. Producers and retailers should avoid using model predictions to make misleading marketing claims. Because the project involves alcohol, the findings should be framed around quality control and education rather than encouraging excessive consumption. The model should support human expertise rather than replace it.

Key Model Summary Tables

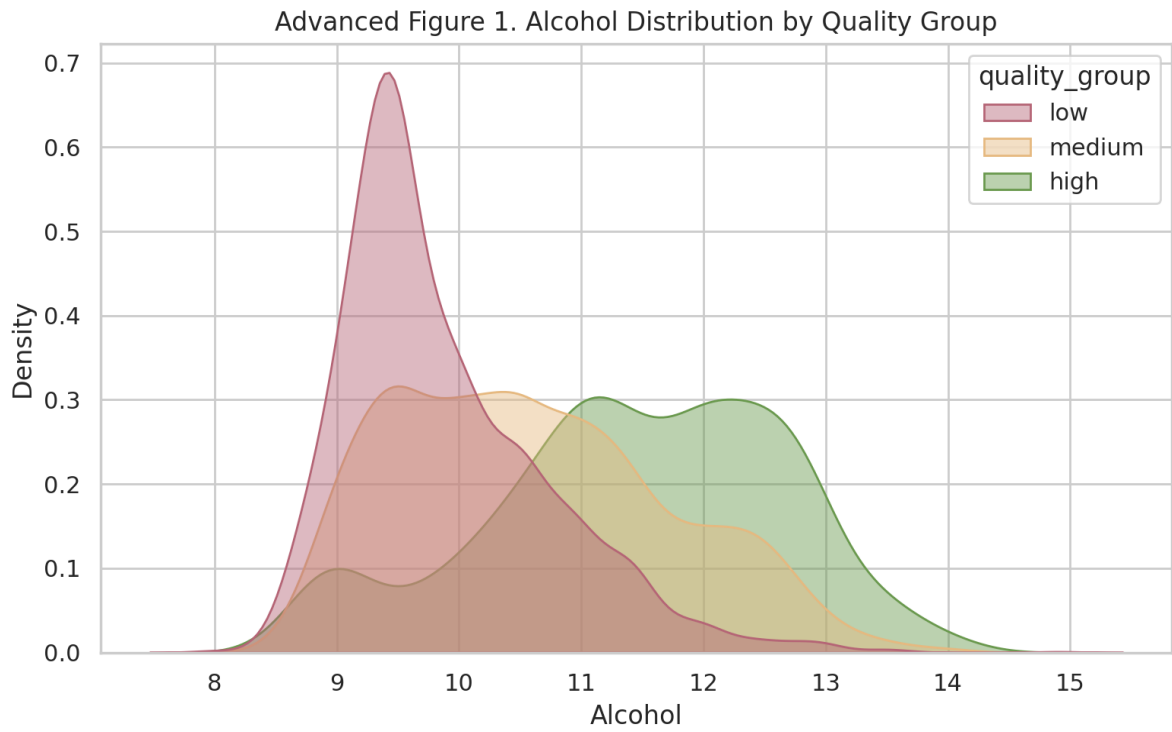
The tables below summarize the most important dataset, baseline, and machine learning results used to support the final conclusions.

Metric	Value
Combined observations	6497
Red wine observations	1599
White wine observations	4898
Missing values	0
Duplicate records flagged	1177
Random Forest classifier accuracy	0.735
Random Forest weighted F1	0.735
Random Forest regression RMSE	0.614
Random Forest regression R-squared	0.493

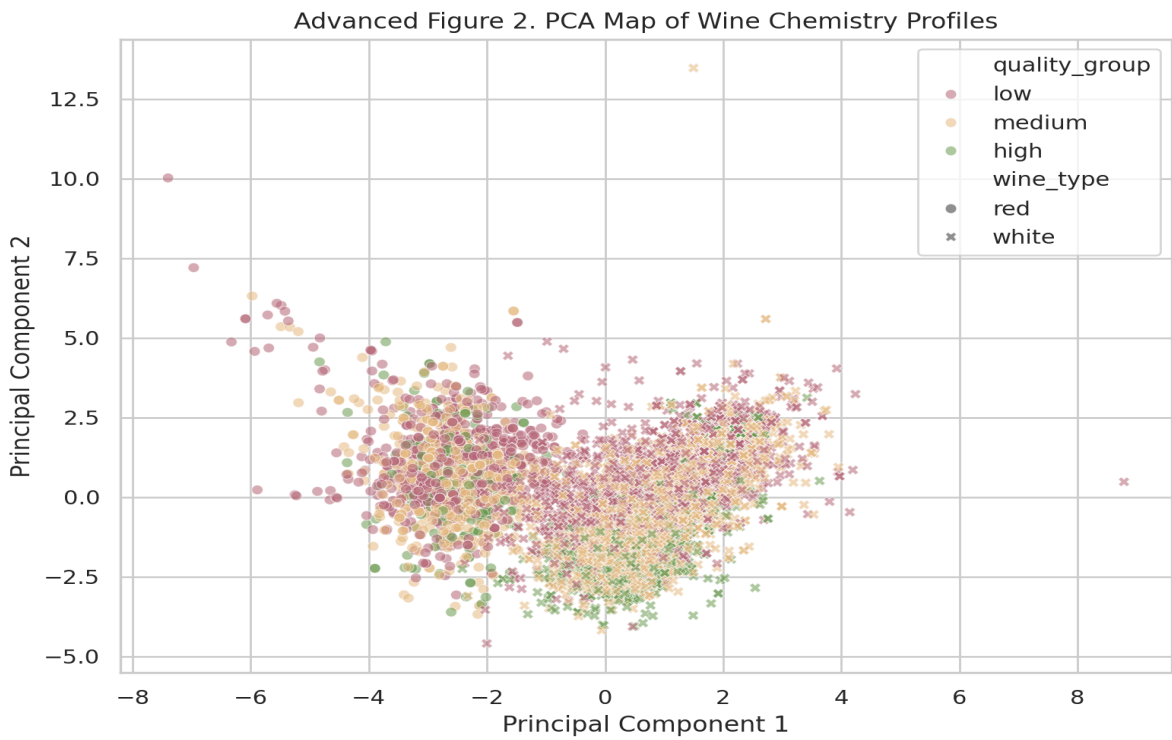
Wine Type	Train RMSE	Test RMSE	Test MAE	Test R-squared
red	0.652	0.627	0.484	0.38
white	0.749	0.757	0.587	0.279

Feature	Importance
alcohol	0.148
density	0.106
volatile acidity	0.1
chlorides	0.087
total sulfur dioxide	0.087
residual sugar	0.082
free sulfur dioxide	0.081
sulphates	0.079
citric acid	0.078
pH	0.078

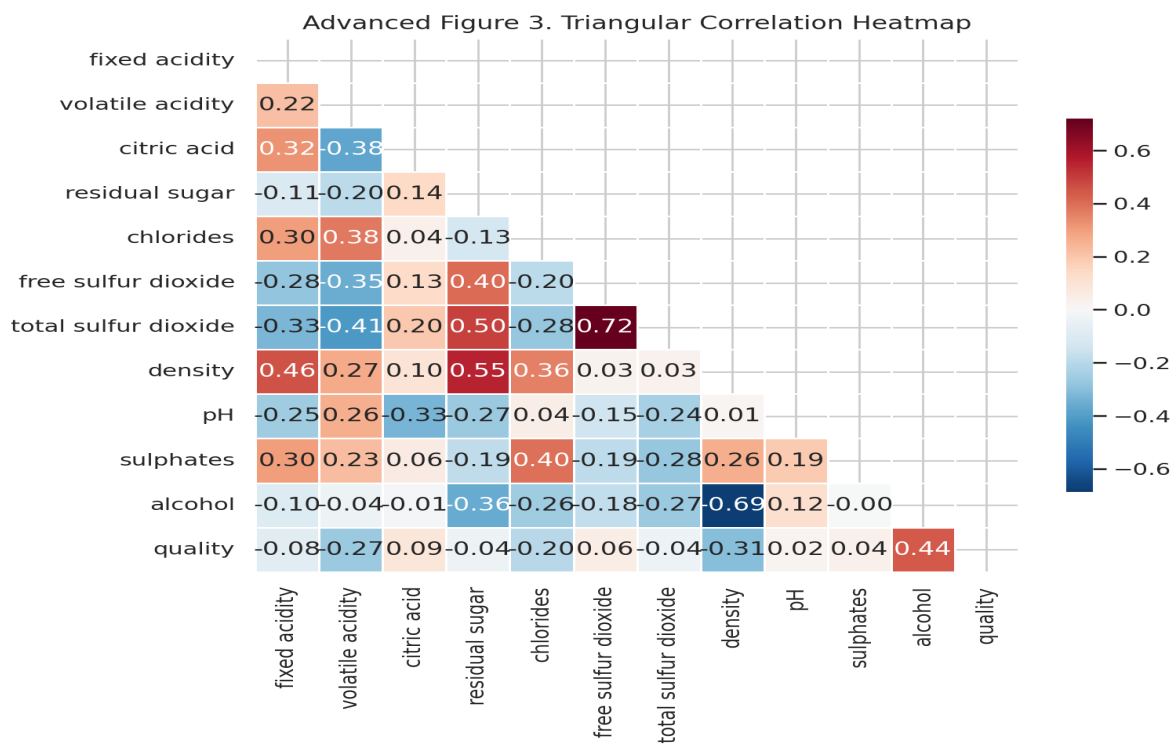
Illustrations



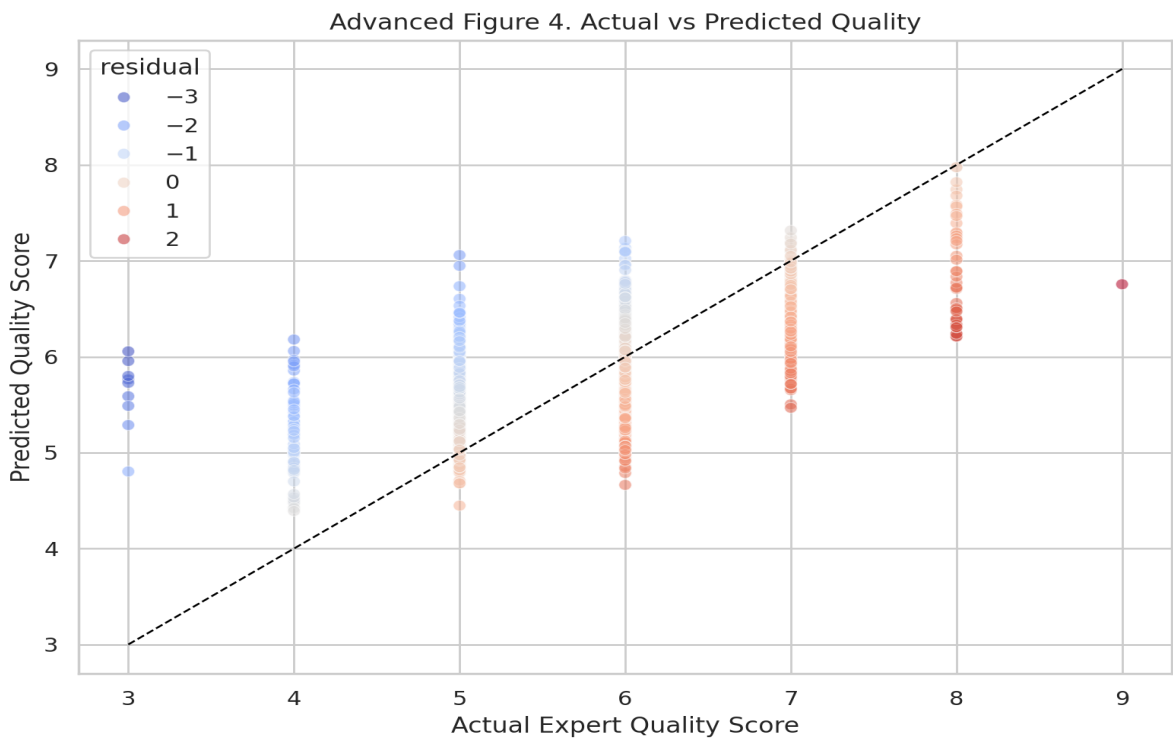
Advanced Figure 1. Alcohol Distribution by Quality Group



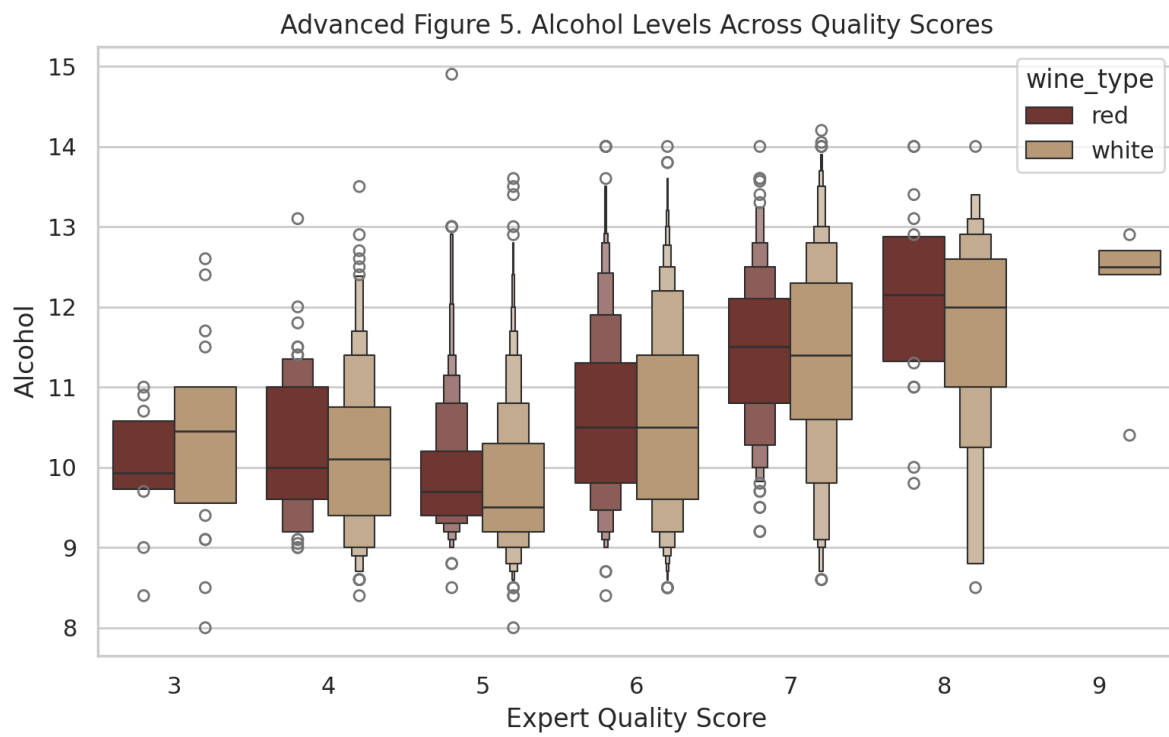
Advanced Figure 2. PCA Map of Wine Chemistry Profiles



Advanced Figure 3. Triangular Correlation Heatmap



Advanced Figure 4. Actual vs Predicted Quality



Advanced Figure 5. Alcohol Levels Across Quality Scores

Appendix A. Supporting Documentation

Dataset overview: The project uses the UCI Wine Quality datasets. The red dataset contains 1,599 observations, and the white dataset contains 4,898 observations. After adding wine_type and combining the data, the final analytical dataset contained 6,497 wine samples.

Data preparation: The two semicolon-delimited CSV files were loaded, labeled by wine type, combined, and reviewed for missing values and duplicate records. No missing values were found. A total of 1,177 duplicate records were flagged for review.

Data dictionary: Original variables include fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, and quality. Added variables were wine_type and quality_group.

Feature engineering: The quality_group field was created for classification and business interpretation. Scores 3 through 5 were classified as low, score 6 as medium, and scores 7 through 9 as high.

Modeling workflow: Separate linear regression models were created for red and white wines. A combined Random Forest classifier predicted quality groups, and a combined Random Forest regressor predicted numeric quality scores.

Audience Questions and Prepared Answers

Question 1: Why did the project analyze red and white wines separately before building a combined model?

Answer: Red and white wines have different chemical profiles, especially in residual sugar, volatile acidity, and sulfur dioxide measures. Separate analysis makes interpretation clearer before using one combined model for prediction.

Question 2: Why was linear regression used as a baseline model instead of only using advanced machine learning methods?

Answer: Linear regression is transparent, easy to explain, and useful as a benchmark. It shows whether more complex models are actually improving performance.

Question 3: How does the Random Forest model improve on the separate red and white linear regression baselines?

Answer: Random Forest can capture nonlinear relationships and feature interactions that linear regression may miss. In this project, the combined Random Forest regressor achieved the lowest RMSE at 0.614.

Question 4: Why was alcohol the strongest predictor of wine quality?

Answer: Alcohol showed the strongest positive correlation with quality in both red and white wines and had the highest Random Forest feature importance. Higher-quality wines in this dataset tended to have higher alcohol levels, although alcohol alone does not determine quality.

Question 5: How should the 1,177 duplicate records be handled before deployment?

Answer: They should be investigated before removal. Some may be true duplicate entries, while others may represent legitimate wines with identical laboratory values. A deployment model should test results with and without duplicates.

Question 6: Does grouping quality into low, medium, and high categories improve usefulness?

Answer: Yes. Exact 0-to-10 prediction is difficult and may imply false precision. Grouping scores into business-friendly categories makes the model easier to use for quality control, product tiering, and decision

support.

Question 7: How reliable is the model when predicting rare high-quality wines rated 8 or 9?

Answer: Reliability is lower for rare classes because there are fewer examples for the model to learn from. This is why class imbalance should be monitored and why results should be evaluated by group, not just overall accuracy.

Question 8: What additional variables would most improve the model?

Answer: Grape variety, vintage, region, winery, weather, fermentation method, aging process, price, sales volume, and consumer reviews would likely improve the model and connect chemistry to market outcomes.

Question 9: How could a winery or retailer use this model without replacing expert judgment?

Answer: The model should be used as decision support. It can flag batches for review, support product tiering, and explain chemical drivers, but final quality decisions should still include expert tasting and business context.

Question 10: What ethical risks exist if predictions are used for marketing?

Answer: The main risk is overstating model predictions as objective proof of quality. Since taste is subjective and alcohol products require responsible communication, predictions should be framed as supportive insights, not absolute claims.

References

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547-553. <https://doi.org/10.1016/j.dss.2009.05.016>
- Dua, D., & Graff, C. (2019). Wine quality data set. UCI Machine Learning Repository. <https://archive.ics.uci.edu/ml/datasets/Wine+Quality>
- Gupta, Y. (2018). Selection of important features and predicting wine quality using machine learning techniques. *Procedia Computer Science*, 125, 305-312. <https://doi.org/10.1016/j.procs.2017.12.041>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Puckette, M. (2020, April 8). How basic wine characteristics help you find favorites. *Wine Folly*. <https://winefolly.com/deep-dive/wine-characteristics/>

milestone_3_wine_quality_analysis_code.py

```
1  # =====
2  # Milestone 3
3  # Adrian Velazquez
4  # DSC 680 Applied Data Science
5  #
6  # Project: Predicting Red and White Wine Quality Using Physicochemical Properties
7  # and Machine Learning
8  #
9  # Data Source: UCI Machine Learning Repository - Wine Quality Dataset
10 # =====
11
12 import os
13 import numpy as np
14 import pandas as pd
15 import matplotlib.pyplot as plt
16 import seaborn as sns
17 from tqdm.auto import tqdm
18 from sklearn.model_selection import train_test_split
19 from sklearn.linear_model import LinearRegression
20 from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
21 from sklearn.compose import ColumnTransformer
22 from sklearn.preprocessing import OneHotEncoder, StandardScaler
23 from sklearn.pipeline import Pipeline
24 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
25 from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
26 from sklearn.decomposition import PCA
27
28 sns.set_theme(style="whitegrid")
29 pd.set_option("display.max_columns", 50)
30
31 RED_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
32 WHITE_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv"
33 RED_FILE = "winequality-red.csv"
34 WHITE_FILE = "winequality-white.csv"
35
36
37 def load_wine_data():
38     """Load red and white wine data from local files or directly from UCI."""
39     red_source = RED_FILE if os.path.exists(RED_FILE) else RED_URL
40     white_source = WHITE_FILE if os.path.exists(WHITE_FILE) else WHITE_URL
41     red_df = pd.read_csv(red_source, sep=";")
42     white_df = pd.read_csv(white_source, sep=";")
43     red_df["wine_type"] = "red"
44     white_df["wine_type"] = "white"
45     wine_df = pd.concat([red_df, white_df], ignore_index=True)
46     wine_df["quality_group"] = pd.cut(
47         wine_df["quality"],
48         bins=[0, 5, 6, 10],
49         labels=["low", "medium", "high"],
50         include_lowest=True
```

```
51     )
52     return red_df, white_df, wine_df
53
54
55 def create_summary_tables(wine_df):
56     """Create core EDA summary tables."""
57     missing_summary_df = wine_df.isna().sum().reset_index()
58     missing_summary_df.columns = ["variable", "missing_count"]
59     missing_summary_df["missing_percent"] = (
60         missing_summary_df["missing_count"] / len(wine_df) * 100
61     ).round(2)
62
63     quality_dist_df = wine_df["quality"].value_counts().sort_index().reset_index()
64     quality_dist_df.columns = ["quality", "count"]
65     quality_dist_df["percent"] = (
66         quality_dist_df["count"] / len(wine_df) * 100
67     ).round(2)
68
69     wine_type_summary_df = wine_df.groupby("wine_type").agg(
70         observations=("quality", "count"),
71         avg_quality=("quality", "mean"),
72         avg_alcohol=("alcohol", "mean"),
73         avg_volatile_acidity=("volatile acidity", "mean"),
74         avg_residual_sugar=("residual sugar", "mean")
75     ).round(3).reset_index()
76
77     quality_group_means_df = wine_df.groupby("quality_group", observed=False)[
78         ["alcohol", "volatile acidity", "density", "chlorides", "sulphates"]
79     ].mean().round(3).reset_index()
80
81     return missing_summary_df, quality_dist_df, wine_type_summary_df, quality_group_means_df
82
83
84 def run_separate_linear_regression_models(wine_df):
85     """Run separate red and white wine linear regression baseline models."""
86     baseline_rows = []
87     correlation_rows = []
88
89     for wine_label in tqdm(["red", "white"], desc="Running baseline models"):
90         model_df = wine_df[wine_df["wine_type"] == wine_label].drop(
91             columns=["wine_type", "quality_group"]
92         )
93         y_vals = model_df["quality"]
94         x_vals = model_df.drop(columns=["quality"])
95         x_train, x_test, y_train, y_test = train_test_split(
96             x_vals,
97             y_vals,
98             test_size=0.25,
99             random_state=3
100         )
101
102         regressor = LinearRegression()
103         regressor.fit(x_train, y_train)
```

```

104     train_pred = regressor.predict(x_train)
105     test_pred = regressor.predict(x_test)
106
107     baseline_rows.append({
108         "wine_type": wine_label,
109         "train_rmse": np.sqrt(mean_squared_error(y_train, train_pred)),
110         "test_rmse": np.sqrt(mean_squared_error(y_test, test_pred)),
111         "test_mae": mean_absolute_error(y_test, test_pred),
112         "test_r2": r2_score(y_test, test_pred)
113     })
114
115     corr_series = model_df.corr(numeric_only=True)["quality"].drop("quality")
116     corr_series = corr_series.sort_values(key=lambda values: values.abs(), ascending=False)
117     for feature_name, corr_value in corr_series.head(5).items():
118         correlation_rows.append({
119             "wine_type": wine_label,
120             "feature": feature_name,
121             "correlation_with_quality": corr_value
122         })
123
124     baseline_results_df = pd.DataFrame(baseline_rows).round(3)
125     separate_correlation_df = pd.DataFrame(correlation_rows).round(3)
126     return baseline_results_df, separate_correlation_df
127
128
129 def run_random_forest_models(wine_df):
130     """Run Random Forest classification and regression models."""
131     feature_columns = [
132         col_name for col_name in wine_df.columns
133         if col_name not in ["quality", "quality_group"]
134     ]
135     x_vals = wine_df[feature_columns]
136     y_quality = wine_df["quality"]
137     y_group = wine_df["quality_group"].astype(str)
138     numeric_features = [col_name for col_name in feature_columns if col_name != "wine_type"]
139
140     preprocess = ColumnTransformer([
141         ("num", StandardScaler(), numeric_features),
142         ("cat", OneHotEncoder(drop="first"), ["wine_type"])
143     ])
144
145     x_train_class, x_test_class, y_train_class, y_test_class = train_test_split(
146         x_vals,
147         y_group,
148         test_size=0.25,
149         random_state=42,
150         stratify=y_group
151     )
152
153     classifier = Pipeline([
154         ("prep", preprocess),
155         ("rf", RandomForestClassifier(
156             n_estimators=250,

```

```
157         random_state=42,
158         class_weight="balanced"
159     ))
160 ])
161 classifier.fit(x_train_class, y_train_class)
162 class_predictions = classifier.predict(x_test_class)
163
164 x_train_reg, x_test_reg, y_train_reg, y_test_reg = train_test_split(
165     x_vals,
166     y_quality,
167     test_size=0.25,
168     random_state=42
169 )
170
171 regressor = Pipeline([
172     ("prep", preprocess),
173     ("rf", RandomForestRegressor(
174         n_estimators=250,
175         random_state=42
176     ))
177 ])
178 regressor.fit(x_train_reg, y_train_reg)
179 reg_predictions = regressor.predict(x_test_reg)
180
181 model_metrics_df = pd.DataFrame([
182     "classification_accuracy": round(accuracy_score(y_test_class, class_predictions), 3),
183     "classification_weighted_f1": round(f1_score(y_test_class, class_predictions, average="weighted"), 3),
184     "regression_rmse": round(np.sqrt(mean_squared_error(y_test_reg, reg_predictions)), 3),
185     "regression_r2": round(r2_score(y_test_reg, reg_predictions), 3)
186 ])
187
188 encoded_feature_names = numeric_features + ["wine_type_white"]
189 feature_importance_df = pd.DataFrame({
190     "feature": encoded_feature_names,
191     "importance": classifier.named_steps["rf"].feature_importances_
192 }).sort_values("importance", ascending=False)
193
194 confusion_matrix_df = pd.DataFrame(
195     confusion_matrix(y_test_class, class_predictions, labels=["low", "medium", "high"]),
196     index=["actual_low", "actual_medium", "actual_high"],
197     columns=["pred_low", "pred_medium", "pred_high"]
198 )
199
200 prediction_results_df = pd.DataFrame({
201     "actual_quality": y_test_reg,
202     "predicted_quality": reg_predictions,
203     "residual": y_test_reg - reg_predictions
204 })
205
206 class_report_text = classification_report(y_test_class, class_predictions)
207 return model_metrics_df, feature_importance_df, confusion_matrix_df, prediction_results_df, class_report_text
208
209
```

```
210 def create_final_visuals(wine_df, prediction_results_df):
211     """Create final advanced visualizations for the white paper."""
212     figure_files = []
213     color_map = {
214         "red": "#7B2D26",
215         "white": "#C19A6B",
216         "low": "#B56576",
217         "medium": "#E6B980",
218         "high": "#6A994E"
219     }
220
221     plt.figure(figsize=(8, 5))
222     sns.kdeplot(
223         data=wine_df,
224         x="alcohol",
225         hue="quality_group",
226         fill=True,
227         common_norm=False,
228         alpha=0.45,
229         palette=[color_map["low"], color_map["medium"], color_map["high"]]
230     )
231     plt.title("Advanced Figure 1. Alcohol Distribution by Quality Group")
232     plt.xlabel("Alcohol")
233     plt.ylabel("Density")
234     plt.tight_layout()
235     fig_name = "advanced_figure_1_alcohol_distribution.png"
236     plt.savefig(fig_name, dpi=200, bbox_inches="tight")
237     figure_files.append(fig_name)
238     plt.show()
239
240     pca_features = wine_df.drop(columns=["quality", "quality_group", "wine_type"])
241     pca_scaled = StandardScaler().fit_transform(pca_features)
242     pca_result = PCA(n_components=2, random_state=42).fit_transform(pca_scaled)
243     pca_plot_df = wine_df[["quality_group", "wine_type"]].copy()
244     pca_plot_df["pc1"] = pca_result[:, 0]
245     pca_plot_df["pc2"] = pca_result[:, 1]
246
247     plt.figure(figsize=(8, 6))
248     sns.scatterplot(
249         data=pca_plot_df,
250         x="pc1",
251         y="pc2",
252         hue="quality_group",
253         style="wine_type",
254         alpha=0.55,
255         s=28,
256         palette=[color_map["low"], color_map["medium"], color_map["high"]]
257     )
258     plt.title("Advanced Figure 2. PCA Map of Wine Chemistry Profiles")
259     plt.xlabel("Principal Component 1")
260     plt.ylabel("Principal Component 2")
261     plt.tight_layout()
262     fig_name = "advanced_figure_2_pca_map.png"
```

```
263 plt.savefig(fig_name, dpi=200, bbox_inches="tight")
264 figure_files.append(fig_name)
265 plt.show()
266
267 corr_df = wine_df.drop(columns=["quality_group", "wine_type"]).corr(numeric_only=True)
268 mask_vals = np.triu(np.ones_like(corr_df, dtype=bool))
269 plt.figure(figsize=(8, 7))
270 sns.heatmap(
271     corr_df,
272     mask=mask_vals,
273     cmap="RdBu_r",
274     center=0,
275     annot=True,
276     fmt=".2f",
277     linewidths=0.5,
278     cbar_kws={"shrink": 0.7}
279 )
280 plt.title("Advanced Figure 3. Triangular Correlation Heatmap")
281 plt.tight_layout()
282 fig_name = "advanced_figure_3_correlation_heatmap.png"
283 plt.savefig(fig_name, dpi=200, bbox_inches="tight")
284 figure_files.append(fig_name)
285 plt.show()
286
287 plt.figure(figsize=(8, 6))
288 sns.scatterplot(
289     data=prediction_results_df,
290     x="actual_quality",
291     y="predicted_quality",
292     hue="residual",
293     palette="coolwarm",
294     alpha=0.55
295 )
296 plt.plot([3, 9], [3, 9], linestyle="--", color="black", linewidth=1)
297 plt.title("Advanced Figure 4. Actual vs Predicted Quality")
298 plt.xlabel("Actual Expert Quality Score")
299 plt.ylabel("Predicted Quality Score")
300 plt.tight_layout()
301 fig_name = "advanced_figure_4_actual_vs_predicted.png"
302 plt.savefig(fig_name, dpi=200, bbox_inches="tight")
303 figure_files.append(fig_name)
304 plt.show()
305
306 plt.figure(figsize=(8, 5))
307 sns.boxenplot(
308     data=wine_df,
309     x="quality",
310     y="alcohol",
311     hue="wine_type",
312     palette={"red": color_map["red"], "white": color_map["white"]}
313 )
314 plt.title("Advanced Figure 5. Alcohol Levels Across Quality Scores")
315 plt.xlabel("Expert Quality Score")
```

```
316     plt.ylabel("Alcohol")
317     plt.tight_layout()
318     fig_name = "advanced_figure_5_alcohol_by_quality.png"
319     plt.savefig(fig_name, dpi=200, bbox_inches="tight")
320     figure_files.append(fig_name)
321     plt.show()
322
323     return figure_files
324
325
326 def main():
327     """Run the full Milestone 3 final analysis."""
328     red_df, white_df, wine_df = load_wine_data()
329     missing_summary_df, quality_dist_df, wine_type_summary_df, quality_group_means_df = create_summary_tables(wine_df)
330     baseline_results_df, separate_correlation_df = run_separate_linear_regression_models(wine_df)
331     model_metrics_df, feature_importance_df, confusion_matrix_df, prediction_results_df, class_report_text = run_random_forest_models(wine_df)
332
333     print("Combined dataset shape")
334     print(wine_df.shape)
335     print("Missing values total")
336     print(int(wine_df.isna().sum().sum()))
337     print("Duplicate rows")
338     print(int(wine_df.duplicated().sum()))
339     print("Quality distribution")
340     print(quality_dist_df)
341     print("Wine type summary")
342     print(wine_type_summary_df)
343     print("Quality group means")
344     print(quality_group_means_df)
345     print("Linear regression baseline results")
346     print(baseline_results_df)
347     print("Separate red and white wine correlations")
348     print(separate_correlation_df)
349     print("Random Forest model metrics")
350     print(model_metrics_df)
351     print("Top Random Forest feature importance")
352     print(feature_importance_df.head(10))
353     print("Classification confusion matrix")
354     print(confusion_matrix_df)
355     print("Classification report")
356     print(class_report_text)
357
358     figure_files = create_final_visuals(wine_df, prediction_results_df)
359     print("Saved figure files")
360     for fig_file in figure_files:
361         print(fig_file)
362
363
364 if __name__ == "__main__":
365     main()
366
```